Unidad

12

Circuitos combinacionales. Álgebra de Boole

El aumento en la velocidad de los componentes digitales y la utilización de un sistema de numeración con dos cifras (0 y 1) ha permitido el avance de materias como la informática, que hoy en día se encuentra al alcance de todos.

Sumar 915 + 814 supone, para nosotros, tres operaciones y un acarreo (9 + 8 es mayor deque 9). Para hacerlo en binario (1110010011 + 1100101110) hay que completar 10 operaciones y 7 acarreos (1 + 1 es mayor deque 2). O sea, de cuatro procesos aritméticos pasamos a realizar un mínimo de 17, con la dificultad añadida de trabajar en un sistema con el que no estamos familiarizados. No obstante, para un ordenador, que realiza millones de operaciones en un segundo, es muy sencillo.

12.1 Sistemas de numeración y códigos

En esta aproximación a los circuitos digitales vamos establecer una primera clasificación basada en: circuitos combinacionales y circuitos secuenciales.

Un **circuito combinacional** es aquel que en cada instante presenta un estado de salida que depende únicamente del estado de sus entradas. Se observa, pues, que un circuito combinacional es, en realidad, una función lógica.

En un **circuito secuencial** —sobre el que profundizaremos en la próxima unidad—, la consideración de sus salidas no sólo depende del estado de las entradas en el momento dado, sino también del historial de las salidas anteriores. Para ello se requiere lo que se denomina *memoria*.

Tanto para unos como para otros es fundamental el sistema de numeración binario y el álgebra de Boole.

A Sistemas de numeración

Los circuitos digitales funcionan mediante la aplicación a sus entradas de señales que toman un número finito de valores.

Un componente físico puede construirse de manera que pueda presentar dos estados diferentes. Por ello, éste es normalmente el número de valores utilizados en las señales conocidas como señales binarias.

Tanto si se utilizan en procesos de datos como en control digital, los sistemas digitales han de realizar operaciones con números, que pueden estar representados en cualquier sistema de numeración y que se van a diferenciar por su base.

El sistema de numeración empleado habitualmente es el que utiliza la base 10 o sistema decimal.

Los circuitos digitales utilizan para su trabajo el sistema de numeración binario, es decir, el que toma como base el número 2.

Por ello, cualquier información que deba ser tratada por un circuito digital ha de ser codificada previamente al sistema que él entiende que es el sistema binario.

La representación de un número N en un sistema de base b, puede realizarse mediante el desarrollo en forma polinómica.

$$N = a_n\, b^n + a_{n-1}\, b^{n-1} + \ldots + a_1\, b^1 + a_0\, b^0 + a_{n-1}\, b^{n-1} \ldots$$

Donde:

b = Base del sistema.

a_i = Coeficientes que representan las cifras del número.

Por ejemplo:

a) Si $b = 10$, $0 \leq a_i < 10$

El número 423,52 se puede expresar como:

$$423{,}52 = 4 \cdot 10^2 + 2 \cdot 10^1 + 3 \cdot 10^0 + 5 \cdot 10^{-1} + 2 \cdot 10^{-2}$$

b) Si $b = 2$, $0 \leq a_i < 2$

El número binario 1101,101 se puede escribir de la siguiente forma:

$$1101{,}101 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3}$$

En los sistemas de numeración no posicionales el valor de un símbolo es siempre el mismo, independientemente del lugar que ocupa; tal es el caso del sistema romano. En un sistema posicional, en cambio, el valor de un símbolo varía según su posición; en el sistema decimal, por ejemplo, se emplean diez dígitos: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, de forma que, todo número mayor de 9, se representa asignando un significado al lugar o posición ocupado por cada dígito. Así, el valor 9 645 se deduce de la suma de los valores que la posición de cada dígito individual le confiere:

$9\,645 = 5 \cdot 10^0 + 4 \cdot 10^1 + 6 \cdot 10^2 + 9 \cdot 10^3$

De esta forma, el dígito de la derecha representa las unidades, el siguiente indica la cantidad de decenas, el siguiente las centenas y así sucesivamente. Notamos que un número se expresa como la suma de potencias de 10 multiplicadas por coeficientes apropiados.

- **Código:** Sistema de signos que permite comprender y formular mensajes.
- **Código binario:** Código que utiliza los signos 0 y 1.
- **Bit:** Unidad mínima de información con dos estados posibles, el 0 y el 1.

B Sistema binario. Código binario natural

Este sistema utiliza para su representación dos símbolos: 0 (cero) y 1 (uno).

A cada uno de estos símbolos se le denomina **bit.** La utilización, casi exclusiva, de este sistema de numeración en sistemas de control y cálculo, se debe a la seguridad y la rapidez de respuesta de los elementos físicos que poseen dos estados diferentes, así como a la sencillez de las operaciones aritméticas en este sistema.

Para pasar un número en sistema binario a su equivalente en sistema decimal, se procede de la siguiente manera: en primer lugar, se expresa el número binario en su polinomio equivalente, y a continuación se opera en el polinomio equivalente, de forma que el resultado obtenido será el número en base 10.

Dado un número binario, se puede determinar el equivalente decimal multiplicando cada bit por su peso y sumar luego todos los valores.

Ejemplo 1

Expresa el número binario $101101{,}11_{(2)}$ en su equivalente decimal.

Solución

$$101101{,}11 = 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2} =$$

$$= 1 \cdot 32 + 0 \cdot 16 + 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 + 1 \cdot \frac{1}{2} + 1 \cdot \frac{1}{2^2} =$$

$$= 45 + 0{,}5 + 0{,}25 = 45{,}75$$

El número binario $101101{,}11_{(2)}$ es equivalente al número decimal $45{,}75_{(10)}$.

Como has visto, cada bit tiene un peso o valor por la posición que ocupa. En la Tabla 12.1 se han transcrito hasta un total de 16 bit de valores enteros y dos decimales.

Bit n.º	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	–1	–2
Peso	2^{15}	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}
Valor	32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1	0,5	0,25

Tabla 12.1. *Peso o valor posicional hasta 16 bits.*

Ejemplo 2

Obtén el número decimal que corresponde al binario del Ejemplo 1 utilizando el sistema de los pesos.

Solución

Peso	32	16	8	4	2	1		1/2	1/4	
Número binario	1	0	1	1	0	1	,	1	1	
Número decimal	32	0	8	4	0	1	,	0,5	0,25	**45,75**

La operación inversa, es decir, pasar un número decimal entero a binario, se lleva a cabo dividiendo sucesivamente entre dos hasta que el último cociente sea inferior a dos. El último cociente será el bit más significativo, seguido de los restos de la división comenzando del último al primero.

Para pasar un número decimal fraccionario a uno binario, se multiplica éste por dos y se toma la parte entera. La parte decimal del número obtenido se vuelve a multiplicar por dos, y el proceso se repite hasta que el resultado sea cero o lleguemos a la precisión necesaria.

Sabías que...

Los ordenadores y demás máquinas que utilizan el sistema binario no pueden utilizar los números exactos tan grandes como quieran, dado que tienen limitado el número de bits para formarlos. Así, tenemos:
Enteros cortos (–32 738 a 32 767) cuando utilizan 16 bits (2 bytes).
Enteros largos (–2 147 483 648 a 2 147 483 647) cuanto utilizan 32 bits (4 bytes).
Precisión simple –3,402823E38 y –1,401298E-45 para valores negativos, y entre 1,401298E-45 y 3 402823E38 para valores positivos. Se utilizan 16 bits.
Precisión doble –1,79769313486232E308 y –4,94065645841247E-324 para valores negativos, y entre 4,94065645841247E-324 y 1,79769313486232E308 para valores positivos. Se utilizan 32 bits.
La representación de los números reales se hace repartiendo los bits en una zona para la parte entera, otra para la decimal y otra para el exponente.

- **Bit más significativo:** Al expresar un número binario, el bit que queda situado más a la izquierda es el denominado de mayor peso o bit más significativo. El que queda más a la derecha recibe el nombre de bit menos significativo.

Sabías que...

El bit más significativo se representa por MSB (*Most Significative Bit*) y el bit menos significativo se representa por LSB (*Least Significative Bit*).

Recuerda

Cuando un número decimal se pasa a binario se dice que está expresado en el código binario natural.

Ejemplo 3

Transforma el número decimal 45 en su equivalente binario.

Solución

	Cociente	Resto
45 : 2	22	1
22 : 2	11	0
11 : 2	5	1
5 : 2	2	1
2 : 2	1	0

bit más significativo → $1\ 0\ 1\ 1\ 0\ 1_{(2)}$

Ejemplo 4

Transforma el número 0,36 en su equivalente binario.

Solución

$0{,}36 \cdot 2 = 0{,}72$ primer dígito fraccionario 0

$0{,}72 \cdot 2 = 1{,}44$ segundo dígito fraccionario 1

$0{,}44 \cdot 2 = 0{,}88$ tercer dígito fraccionario 0

$0{,}88 \cdot 2 = 1{,}76$ cuarto dígito fraccionario 1

El resultado es el siguiente para la aproximación realizada:

$0{,}36_{(10)} = 0{,}0101_{(2)}$

Ejemplo 5

Obtén el número equivalente binario del número decimal 132,63.

Solución:

Primero se halla el equivalente de la parte entera.

	Cociente	Resto
132 : 2	66	0
66 : 2	33	0
33 : 2	16	1
16 : 2	8	0
8 : 2	4	0
4 : 2	2	0
2 : 2	1	0

bit más significativo → $1\ 0\ 0\ 0\ 0\ 1\ 0\ 0$

A continuación, se obtiene el equivalente aproximado de la parte decimal.

$$0{,}63 \cdot 2 = 1{,}26$$

$$0{,}26 \cdot 3 = 0{,}52$$

$$0{,}52 \cdot 2 = 1{,}04$$

Por tanto, el número 132,63 tendrá como equivalente binario:

$$132{,}63_{(10)} = 10000100{,}101_{(2)}$$

C Sistema hexadecimal

Es un sistema de numeración muy empleado en microprocesadores y, por tanto, conviene describir sus características más generales. La razón fundamental para utilizar este código es la facilidad de conversión con el código binario.

El sistema hexadecimal es el que tiene base 16. Para su representación se utilizan los diez primeros dígitos decimales (del 0 al 9) y las letras del alfabeto (A, B, C, D, E y F).

En la Tabla 12.2 se muestra la equivalencia entre los sistemas hexadecimal, decimal y binario.

Para pasar un número binario a hexadecimal, en primer lugar se hacen grupos de cuatro bits, partiendo de la coma hacia la izquierda y hacia la derecha. Si el último grupo de cada lado de la coma está incompleto, se añaden ceros por la izquierda o por la derecha, según estemos a un lado o a otro de la coma. En segundo y último lugar, se realiza la equivalencia entre el número binario y el hexadecimal correspondiente, según la Tabla 12.2.

Vamos a ver con los siguientes ejemplos la conversión de información binaria a hexadecimal y viceversa.

Ejemplo 6

Transforma el número binario 10111011101,101101 en hexadecimal.

Solución

Se realizan grupos de cuatro bits, partiendo de la coma a izquierda y derecha:

0101 . 1101 . 1101 , 1011 . 0100

Se observa que se ha añadido un cero a la izquierda de la coma y dos ceros a la derecha, para completar grupos de 4 bits.

Se convierte cada grupo de cuatro bits en su equivalente hexadecimal:

5 D D , B 4

Ejemplo 7

Convierte el número 34AF,D8 en binario.

Solución

Se procede a la inversa que en el problema anterior.

3	4	A	F	,	D	8
0011	0100	1010	1111	,	1101	1000

Para pasar de decimal a hexadecimal o viceversa se actúa de la forma explicada para el código binario natural, pero, en este caso, se divide entre 16 o se multiplica, según proceda.

Ejemplo 8

Pasar el número $4735_{(10)}$ a hexadecimal y viceversa.

Solución

	C	R
4735 : 16	295	15 (F)
295 : 16	18	7
18 : 16	1	2

$4735_{(10)} = 127F_{(H)}$

Para pasar el número $127F_{(H)}$ a decimal se procede de la siguiente forma:

$127F_{(H)} = 1 \cdot 16^3 + 2 \cdot 16^2 + 7 \cdot 16^1 + 15 \cdot 16^0 = 4096 + 512 + 112 + 15 = 4735_{(10)}$

D	H	B
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Tabla 12.2. *Tabla de equivalencia entre los códigos decimal, hexadecimal y binario.*

D	O	B
0	0	000
1	1	001
2	2	010
3	3	011
4	4	100
5	5	101
6	6	110
7	7	111

Tabla 12.3. *Equivalencia entre los sistemas decimales, octal y binario.*

Decimal	BCD Natural	BCD Aiken	BCD Ex-3
0	0000	0000	0011
1	0001	0001	0100
2	0010	0010	0101
3	0011	0011	0110
4	0100	0100	0111
5	0101	1011	1000
6	0110	1100	1001
7	0111	1101	1010
8	1000	1110	1011
9	1001	1111	1100

Tabla 12.4. *Equivalencia entre los códigos BCD.*

D Códigos binarios

Un código es una representación de cantidades, de tal forma que a cada una de éstas se le asigna una combinación de símbolos determinada, y viceversa.

Dentro de los códigos binarios, los más utilizados son los códigos BCD (*decimal codificado en binario*).

Debido a que con n cifras binarias o bits se pueden obtener 2^n combinaciones diferentes, a cada combinación se le puede asignar una cantidad diferente.

Para codificar un número decimal mediante este sistema, cada una de sus cifras se representa por separado. El número de bits necesarios para representar cada cifra es de cuatro y, por tanto, se pueden efectuar $2^4 = 16$ combinaciones distintas.

El código más utilizado es el código *BCD natural*, que emplea las diez primeras combinaciones en orden creciente. También existen otros códigos de la familia BCD, como son: el *código Aiken*, que emplea las cinco primeras combinaciones y las cinco últimas de las 16 que pueden formarse con cuatro bits; y el *exceso tres*, que no toma ni las tres primeras ni las tres últimas, sino las 10 que quedan en medio. La Tabla 12.4 muestra la correspondencia entre los distintos códigos de la familia BCD y el sistema decimal.

Otra clase de códigos binarios son los denominados *códigos progresivos*, cuya característica principal es que una combinación difiere de la anterior y de la siguiente en un solo bit, lo que facilita la detección de errores y la transmisión de datos.

Los *códigos detectores y correctores de error* son códigos más complejos que detectan y corrigen el error en la información binaria. Los más conocidos son los *códigos de paridad*, representados en la Tabla 12.5, y el *código dos entre cinco o biquinario*.

También hay otros tipos de códigos que se emplean para presentar información de letras, números y símbolos especiales, entre los que destacan el *ASCII* (*American Standard Code for Information Interchange*) y que se encuentra en cualquier manual de ordenador o de impresora.

Sabías que...

El código BCD natural asigna a cada posición los pesos 8 4 2 1, y que por ello también es conocido como código 8421. El código BCD Aiken asigna a cada posición los pesos 2 4 2 1, y que por ello es conocido como código Aiken 2421.

0	00000
1	00011
2	00101
3	00110
4	01001
5	01010
6	01100
7	01111
8	10001
9	10010

Tabla 12.5. *Código de paridad par formado a partir de código BCD natural. Se añade 0 o 1 a la derecha del número binario según la cantidad de «unos» de éste. Si el número de «unos» es par se añade un cero y si es impar, un uno.*

Recuerda

El código ASCII es un código alfanumérico, ya que se utiliza para representar números, letras o símbolos.

Ejemplo 9

Codifica el número decimal 342,75 utilizando los distintos códigos BCD.

Solución

a) BCD natural.

$342{,}75_{(10)} = 0011\ \ 0100\ \ 0010,\ \ 0111\ \ 0101$

$\qquad\qquad\quad\ \ 3\ \ \ \ 4\ \ \ \ 2\ \ \ \ 7\ \ \ \ 5$

b) BCD Aiken

$342{,}75_{(10)} = 0011\ \ 0100\ \ 0010,\ \ 1101\ \ 1011$

$\qquad\qquad\quad\ \ 3\ \ \ \ 4\ \ \ \ 2\ \ \ \ 7\ \ \ \ 5$

c) BCD Exceso tres.

$342{,}75_{(10)} = 0110\ \ 0111\ \ 0101,\ \ 1010\ \ 1000$

$\qquad\qquad\quad\ \ 3\ \ \ \ 4\ \ \ \ 2\ \ \ \ 7\ \ \ \ 5$

12.2 Álgebra de Boole. Definiciones

En 1847 el matemático inglés George Boole desarrolló un álgebra que lleva su nombre, y cuyo objetivo consiste en representar las formas de razonamiento lógico, sistematizarlas y profundizar en el conocimiento de sus mecanismos. En sus formulaciones originales, el **álgebra de Boole** manejaba variables que representaban posiciones que podían adoptar dos valores: verdadero y falso.

En 1948 Claude Shamon adaptó el álgebra de Boole al estudio de los circuitos que contienen elementos que adoptan dos estados estables.

El álgebra de Boole opera con variables que admiten únicamente dos valores, que se designan por 0 y 1 (cero y uno). Estos símbolos no representan números, sino dos estados diferentes de un dispositivo, es decir, una lámpara puede estar **encendida** (1) o **apagada** (0), un interruptor puede estar **abierto** (0) o **cerrado** (1).

Sabías que...

Para no tener que distinguir entre lógica positiva o negativa, los fabricantes de dispositivos digitales no hablan de ceros y unos, sino de niveles de tensión alto (H, *High*) y bajo (L, *Low*).

A Lógica de niveles

En los circuitos digitales es necesario adoptar un sistema de representación para diferenciar los estados lógicos 0 y 1 (cero y uno). Para ello, se utiliza el procedimiento denominado **lógica de niveles,** que establece una correspondencia entre los niveles de tensión y los elementos de información binaria.

Existen dos tipos de lógica, **positiva** y **negativa.**

En lógica positiva, al nivel de tensión más elevado se le asigna el estado 1 (uno), mientras que al nivel más bajo se le establece el estado lógico 0 (cero), tal y como se puede observar en la Figura 12.1.

Por el contrario, en lógica negativa el nivel más bajo de tensión se corresponde con el estado lógico 1 (uno), mientras que al nivel más alto se le asocia el estado lógico 0 (cero), tal y como se muestra en la Figura 12.2.

Al hablar de funciones u operaciones lógicas hay que tener presente el tipo de lógica en el que se está trabajando, si es positiva o negativa. A partir de ahora, nosotros utilizaremos la lógica positiva.

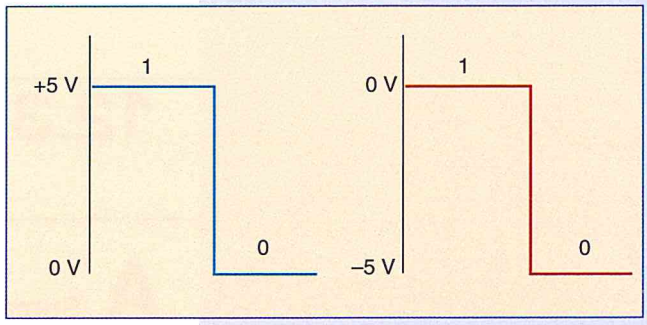


Fig. 12.1. *Lógica positiva.*

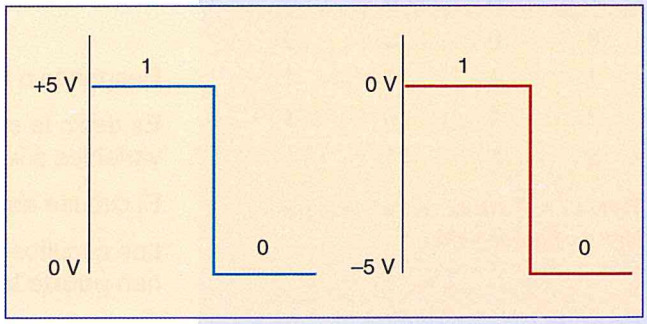


Fig. 12.2. *Lógica negativa.*

B Variables y funciones lógicas. Tablas de verdad

Antes de seguir con las operaciones, vamos a definir qué es una variable lógica, qué es una tabla de verdad y qué es una función lógica.

Para nosotros una proposición es una afirmación exenta de ambigüedad que se manifiesta por medio de una oración gramatical y que vamos a representar mediante un símbolo (una letra, generalmente) que será la **variable lógica.** Así:

Pulsar el botón verde = *a*, será una proposición y *a* la variable que la representa.

a
0
1

Tabla 12.6. *Valores lógicos para una variable.*

b	a
0	0
0	1
1	0
1	1

(a)

b	a	S
0	0	0
0	1	0
1	0	0
1	1	1

(b)

Tabla 12.7. *a) Posibilidades de valores de dos variables. b) Tabla de verdad de la función lógica $S = a \cdot b$.*

Los estados de la proposición o de la variable lógica (se pueden considerar sinónimos) pueden ser dos, verdadero (1, botón pulsado) o falso (0, botón sin pulsar), como se mestra en la Tabla 12.6. A la disposición de la Tabla 12.6 se le denomina *tabla de verdad de la variable*.

Se pueden combinar proposiciones o variables lógicas. De forma que pulsar el botón verde (variable *a)* y poner la llave de contacto (variable *b*) indique la puesta en marcha de un motor (variable *S*). Esta última variable es dependiente de las anteriores, que serán independientes, y se denomina **función lógica** (igual que las funciones habituales en matemáticas). Se puede escribir $S = a \cdot b$, significando el punto la operación de que se realicen a la vez las dos proposiciones o variables lógicas. Ahora, las posibilidades con dos variables independientes son cuatro (Tabla 12.7a).

La tabla de verdad de la función lógica *S* (ponerse en marcha el motor) solamente será afirmativa en el caso de que lo sean simultáneamente las dos variables lógicas independientes(Tabla 12.7b). A esta última tabla se le denomina **tabla de verdad de la función lógica.**

Si hubiera tres variables independientes, las posibilidades serían 8 y, en general 2^n, donde *n* es el número de proposiciones o variables lógicas.

Observa que para la formación de las combinaciones de la tabla de verdad se ha numerado en binario (compara con la Tabla 12.2 donde se encuentran representados los dieciséis primeros números binarios).

12.3 Operaciones básicas en el álgebra de Boole

A Suma lógica o unión (puerta OR)

Se trata de una función lógica que se representa (suponiendo las variables de entrada *a* y *b)* de la siguiente manera:

$$S = b + a$$

Responde a la Tabla de verdad 12.8.

	b	a	S
0	0	0	0
1	0	1	1
2	1	0	1
3	1	1	1

Tabla 12.8. *Tabla de verdad de la suma lógica o función unión.*

Es decir, la suma lógica $S = b + a$ toma el valor lógico 1 cuando *a* o *b*, o las dos variables a la vez, tienen 1 como valor.

El circuito equivalente realizado con interruptores se muestra en la Figura 12.3.

Los circuitos electrónicos que realizan directamente esta operación se denominan **puerta lógica O (OR)** o simplemente puerta O.

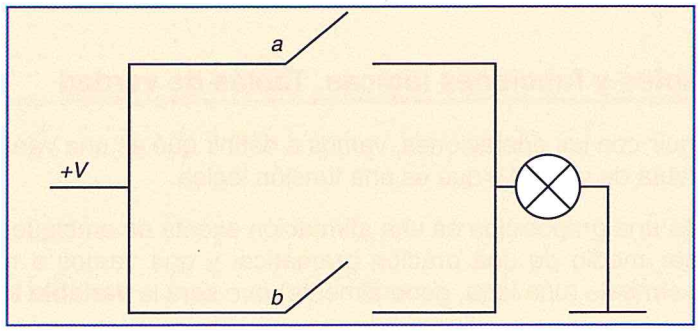


Fig. 12.3. *Esquema eléctrico correspondiente a la suma lógica. La lámpara se enciende si al menos uno de los dos interruptores están cerrados.*

Los símbolos que se utilizan para su representación pueden ser, según las normas empleadas, los que se muestran en la Figura 12.4.

Comercialmente, en tecnología TTL (*Transistor-Transistor-Logic*) existe el circuito 7432, que se compone de cuatro puertas OR con dos entradas. Es un circuito integrado por 14 patillas o pines, y las cuatro puertas están dispuestas como se muestra en la Figura 12.5.

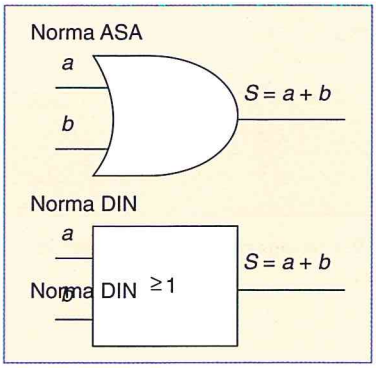


Fig. 12.4. *Puerta OR según las normas ASA y DIN.*

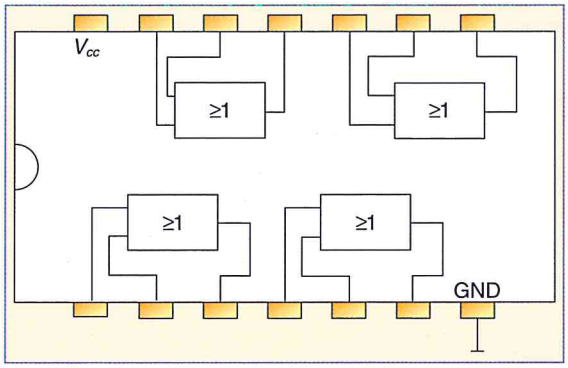


Fig. 12.5. *Circuito integrado 7432.*

B Producto lógico o función intersección (puerta AND)

Se trata de una función lógica que, aplicada a dos variables de entrada *a* y *b,* se representa como:

$$S = a \cdot b$$

y responde a la Tabla de verdad 12.9.

Es decir, el producto lógico $a \cdot b$ toma el valor lógico 1 cuando *a* y *b* presentan 1.

El circuito eléctrico equivalente responde a la asociación de dos interruptores en serie (Fig. 12.6).

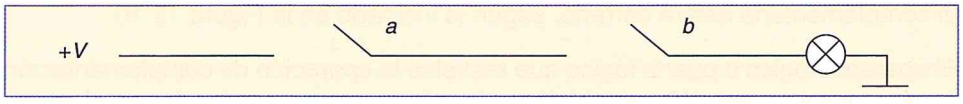


Fig. 12.6. *Símil eléctrico del producto lógico. La lámpara se enciende sólo si ambos interruptores están cerrados.*

Al igual que sucede con la suma lógica, en el producto lógico existen circuitos electrónicos que realizan esta operación directamente, y se denominan **puertas lógicas Y (AND).**

El símbolo empleado para representarlas puede ser, siguiendo la norma utilizada, el indicado en la Figura 12.7.

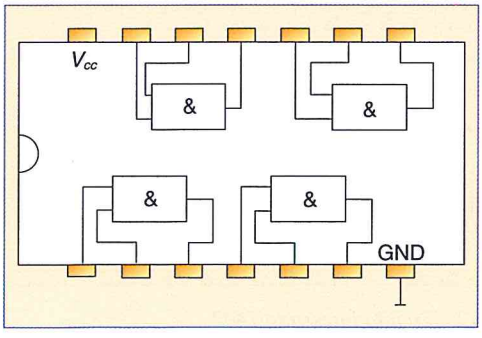


Fig. 12.8. *Circuito integrado 7408.*

Desde el punto de vista comercial, existen varios circuitos integrados diseñados en tecnología TTL, que están formados por puertas AND; por ejemplo:

- 7408, cuatro puertas AND de dos entradas, representado en la Figura 12.8.
- 7411, tres puertas AND de tres entradas.
- 7421, dos puertas AND de cuatro entradas.

	b	a	S
0	0	0	0
1	0	1	0
2	1	0	0
3	1	1	1

Tabla 12.9. *Tabla de verdad del producto lógico o función intersección.*

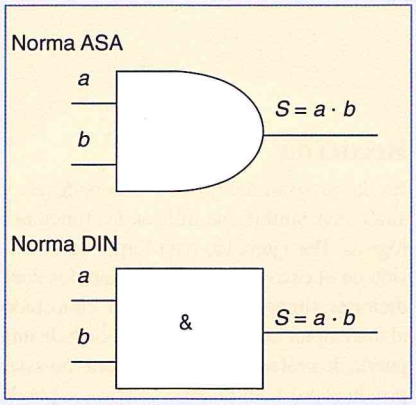


Fig. 12.7. *Puente AND según las normas ASA y DIN.*

C Función igualdad

La función de igualdad es la más sencilla de todas, pues en ella sólo interviene una variable. Su expresión matemática es:

$$S = a$$

y responde a la Tabla de verdad 12.10 y al circuito eléctrico de la Figura 12.9. Esta función no se realiza con ninguna puerta lógica.

	a	*S*
0	0	0
1	1	1

Tabla 12.10. *Tabla de verdad de la función igualdad.*

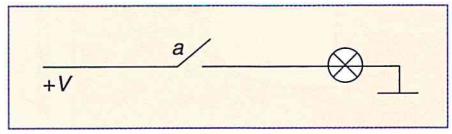


Fig. 12.9. *Circuito eléctrico de la función igualdad.*

D Complementación o función negación (puerta NOT o inversora)

Esta función es conocida también como **función NO.** Se aplica a una sola variable de entrada, de tal forma que si ésta es, por ejemplo, *a*, su complementación se representa por $\overline{a}$ (se lee «*a* negada») y responde a la Tabla de verdad 12.11.

$$S = \overline{a}$$

Es decir, el complemento de una variable equivale a invertir el estado lógico anterior. Por ello, esta función también se conoce como inversora.

En circuitos eléctricos, esta operación se representa con dos contactos complementarios de un mismo interruptor, con la condición de que, si uno está abierto, el complementario estará cerrado, según lo indicado en la Figura 12.10.

El operador lógico o puerta lógica que resuelve la operación de complementación se denomina **puerta NO** y su símbolo, según la norma empleada, es el representado en la Figura 12.11.

En el mercado existe el circuito integrado 7404, que contiene seis inversores, tal y como vemos en la Figura 12.12.

	a	*S*
0	0	1
1	1	0

Tabla 12.11. *Tabla de verdad de la función negación.*

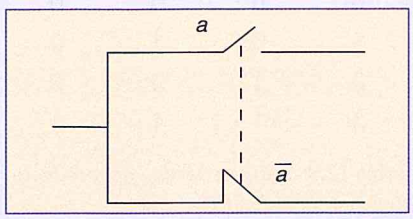


Fig. 12.10. *Simil eléctrico de la función negación. Si* a *se cierra,* ā *se abrirá y viceversa.*

Recuerda

En la programación con ordenadores y autómatas también se utilizan las funciones lógicas. Por ejemplo, para tomar una decisión en el caso de que se cumplan dos condiciones simultánemente (estar conectado el interruptor de marcha y estar cerrada una puerta de protección) y una tercera (no estar pisado el pulsador de emergencia), se puede expresar así:

If (a AND b) AND (NOT c) then SET m.

En la programación disponemos solamente de letras, por lo que se tienen que utilizar las palabras reservadas para representar las operaciones lógicas elementales. Cada una de las letras, a, b, c, m, indican entradas o salidas del autómata o el valor de una variable del programa informático.

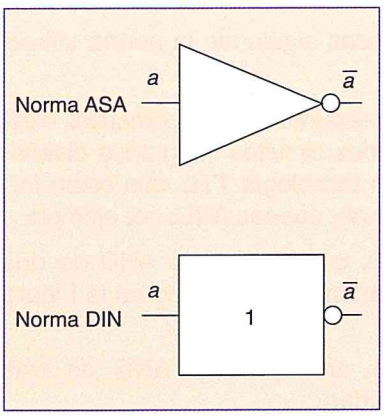


Fig. 12.11. *Puerta NO según las normas ASA y DIN.*

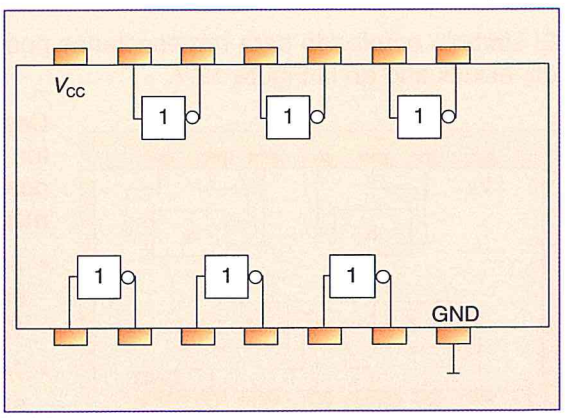


Fig. 12.12. *Circuito integrado 7402.*

12.4 Postulados, propiedades y teoremas del álgebra de Boole

En el apartado anterior señalábamos que las operaciones en las que está basada el álgebra de Boole son la suma (+) y el producto (·) lógicos, así como la función negación. Se verifican las propiedades, los postulados y los teoremas siguientes.

Propiedades

- Propiedad conmutativa:

$$a + b = b + a$$
$$a \cdot b = b \cdot a$$

- Propiedad asociativa:

$$a + b + c = a + (b + c)$$
$$a \cdot b \cdot c = a \cdot (b \cdot c)$$

- Propiedad distributiva:

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$$
$$a + (b \cdot c) = (a + b) \cdot (a + c)$$

Postulados

$$a + 1 = 1 \qquad a \cdot 1 = a$$
$$a + 0 = a \qquad a \cdot 0 = 0$$
$$a + a = a \qquad a \cdot a = a$$
$$a + \overline{a} = 1 \qquad a \cdot \overline{a} = 0$$
$$\overline{\overline{a}} = a$$

La variable $\overline{a}$ es el complemento de a. Las dos variables se encuentran siempre en un estado binario contrario; es decir, si $a = 1 \Rightarrow \overline{a} = 0$, y viceversa.

Teoremas

- $a + a \cdot b = a$

  Para demostrar esta igualdad se saca factor común de la variable a:

  $a + a \cdot b = a\,(1 + b)$; *como* $(1 + b) = 1$, *resulta:* $a + a \cdot b = a \cdot 1 = a$

- $a \cdot (a + b) = a$

  Demostración:

  $a \cdot (a + b) = a$, aplicando la propiedad distributiva:

  $a \cdot (a + b) = a \cdot a + a \cdot b = a + a \cdot b = a$

- $a + \overline{a} \cdot b = a + b$

  Comprobación:

  $a + \overline{a} \cdot b = (a + \overline{a}) \cdot (a + b) = 1 \cdot (a + b) = a + b$

- $b \cdot (a + \overline{b}) = a \cdot b$

  Demostración:

  $b \cdot (a + \overline{b}) = b \cdot a + b \cdot \overline{b} = b \cdot a + 0 = b \cdot a = a \cdot b$

- Teoremas de Morgan:

$$\overline{a + b} = \overline{a} \cdot \overline{b} \qquad \overline{a \cdot b} = \overline{a} + \overline{b}$$

  La demostración puede verse al margen.

Sabías que...

Para representar las variables binarias por contactos o interruptores las variables directas se representan por un contacto abierto. Si la variable está negada, por un contacto cerrado.

Sabías que...

Los teoremas de Morgan se pueden demostrar por la tabla de verdad.

1. $\overline{a + b} = \overline{a} \cdot \overline{b}$

b	a	$\overline{a + b}$	$\overline{a} \cdot \overline{b}$
0	0	1	1
0	1	0	0
1	0	0	0
1	1	0	0

2. $\overline{a \cdot b} = \overline{a} + \overline{b}$

b	a	$\overline{a \cdot b}$	$\overline{a} + \overline{b}$
0	0	1	1
0	1	1	1
1	0	1	1
1	1	0	0

12.5 Otras funciones lógicas

	b	a	S
0	0	0	1
1	0	1	0
2	1	0	0
3	1	1	0

Tabla 12.12. *Tabla de verdad de la función NOR.*

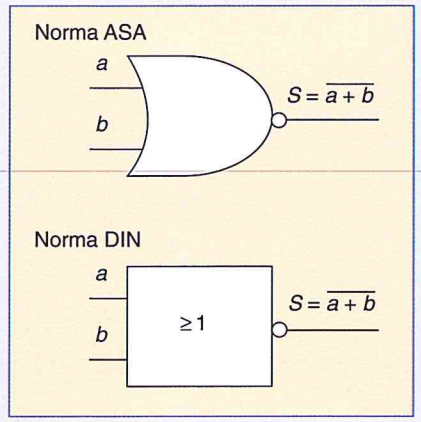


Fig. 12.13. *Símbolos de la puerta NOR.*

A Función o puerta NOR

Esta función se obtiene al invertir la salida de una función OR. El resultado es la negación de la función suma, es decir, una función NO OR, o de forma abreviada, función NOR.

Para dos variables, su expresión matemática es:

$$S = \overline{a + b}$$

Si se aplica el teorema de Morgan, resulta:

$$S = \overline{a} \cdot \overline{b}$$

La tabla de verdad de la puerta NOR es la Tabla 12.12.

El símbolo depende, como siempre, de la norma utilizada. En la Figura 12.13 se muestran los más usuales.

En el mercado existen los siguientes circuitos integrados con tecnología TTL:

- 7402, cuatro puertas NOR de dos entradas.
- 7427, tres puertas NOR de tres entradas.
- 74260, dos puertas NOR de cinco entradas.

En la Figura 12.14 se puede observar la disposición de puertas NOR que toma el circuito integrado 7402.

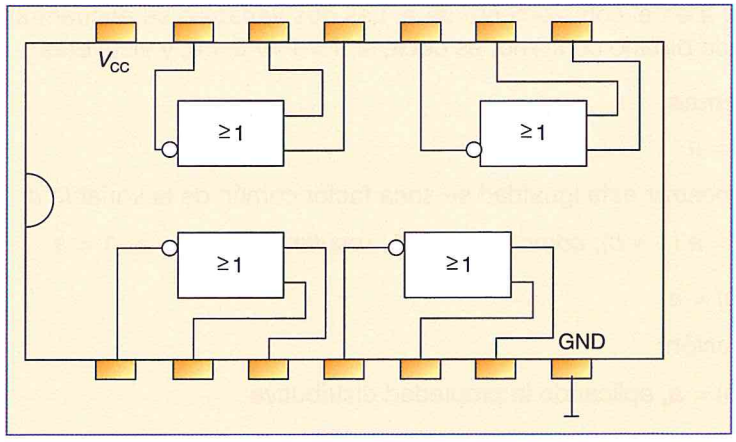


Fig. 12.14. *Circuito integrado 7402.*

	b	a	S
0	0	0	1
1	0	1	1
2	1	0	1
3	1	1	0

Tabla 12.13. *Tabla de verdad de la puerta NAND.*

B Función o puerta NAND

Realiza el producto lógico u operación Y, y después la complementa, es decir, realiza la función NO-AND, que, abreviadamente, se representa como función NAND.

La salida de la función NAND es, si *a* y *b* son las variables de entrada:

$$S = \overline{a \cdot b}$$

que, al aplicar el teorema de Morgan, también se puede expresar:

$$S = \overline{a} + \overline{b}$$

La Tabla 12.13 es la tabla de verdad de una puerta NAND.

Los símbolos más empleados se pueden ver en la Figura 12.15.

En el mercado existen los siguientes circuitos integrados con tecnología TTL:

- 7400, cuatro puertas NAND de dos entradas.
- 7410, tres puertas NAND de tres entradas.
- 7420, dos puertas NAND de cuatro entradas.
- 7430, puerta NAND de ocho entradas.

En la Figura 12.16 aparece la disposición interna del circuito 7400.

A las puertas NOR y NAND se les denomina también **puertas universales,** debido a que todas las funciones lógicas se pueden formar a partir de ellas, tal y como se analizará más adelante.

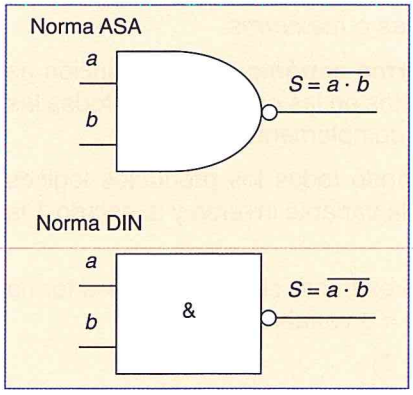


Fig. 12.15. *Símbolos de la puerta NAND.*

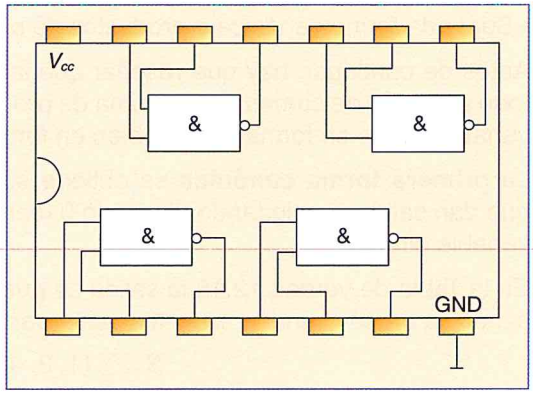


Fig. 12.16. *Circuito integrado 7400.*

C Función o puerta O-Exclusiva

La puerta O-Exclusiva, de dos variables *a* y *b*, se define como aquella que presenta a su salida el valor 1 cuando las variables de entrada no coinciden, y 0 (cero) cuando el valor de las variables de entrada coincide, tal como muestra la Tabla 12.14, que representa la tabla de verdad correspondiente a esta puerta.

$$S = a \oplus b = \overline{b} \cdot a + b \cdot \overline{a}$$

	b	a	S
0	0	0	0
1	0	1	1
2	1	0	1
3	1	1	0

Tabla 12.14. *Tabla de verdad de la puerta EXOR.*

Esta puerta lógica también presenta una función equivalente a un comparador binario de un bit, e indica si dos bits son iguales o diferentes.

Los símbolos son los que aparecen en la Figura 12.17, que, como siempre, representan los símbolos de las normas más utilizadas.

Comercialmente, y en tecnología TTL, se puede encontrar el circuito integrado 7486, que contiene cuatro puertas EXOR de dos entradas (Fig. 12.18).

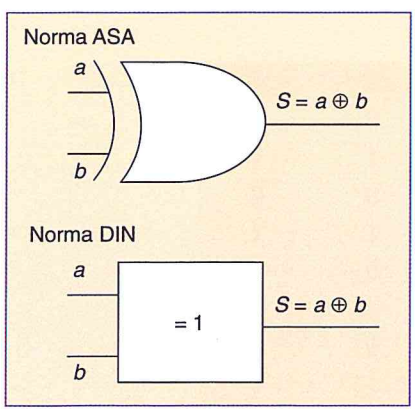


Fig. 12.17. *Símbolos de la puerta EXOR.*

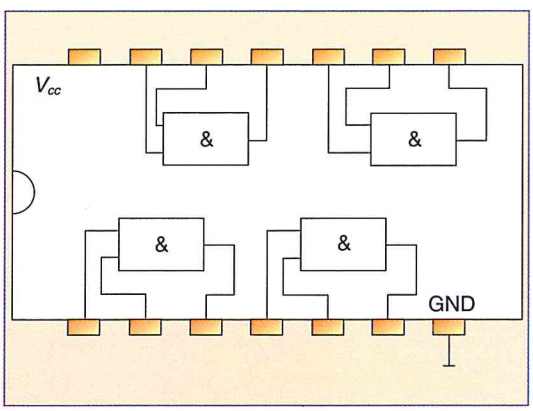


Fig. 12.18. *Circuito integrado 7486.*

12.6 Obtención de la función lógica a partir de la tabla de verdad

Para la resolución de circuitos lógicos combinacionales, normalmente se sigue un proceso cuyo primer paso consiste en la confección de una tabla de verdad que establece todas las combinaciones posibles de entrada y determina, para cada una de ellas, el estado de la salida según el planteamiento del problema.

A partir de la tabla de verdad se puede obtener la función lógica de dos maneras distintas:

- Primera forma canónica o suma de productos o *minterms*.
- Segunda forma canónica o productos de sumas o *maxterms*.

Antes de continuar, hay que reseñar que la **forma canónica** de una función es todo producto de sumas o toda suma de productos en las que aparecen todas las variables, bien en forma directa, bien en forma complementada.

La **primera forma canónica** se obtiene sumando todos los productos lógicos que dan salida 1, asignando al estado 0 (cero) la variable inversa y al estado 1 la variable directa.

En la Tabla de verdad 12.15 la salida se puede expresar según la primera forma canónica como se indica a continuación para $n = 3$ variables:

$$S = \sum_{3} (1, 3, 4, 6, 7)$$

$$S = \overline{c} \cdot \overline{b} \cdot a + \overline{c} \cdot b \cdot a + c \cdot \overline{b} \cdot \overline{a} + c \cdot b \cdot \overline{a} + c \cdot b \cdot a$$

Sabías que...

Otra forma de obtener la segunda forma canónica consiste en contar la tabla de verdad a la inversa; es decir, la combinación 7 será la 0 y la 0, la 7.

Para deducir la **segunda forma canónica,** producto de sumas, se puede seguir este razonamiento: la salida en forma de productos de sumas ($S = \prod_{3}$) será igual a $2^n - 1 - i$, donde i es el lugar que ocupan los términos que dan salida 0 (cero).

En la Tabla de verdad 12.15 las combinaciones que hacen S = 0 son las combinaciones 0, 2 y 5.

$$\begin{aligned} i &= 0 \qquad 7 - 0 = 7 \\ i &= 2 \qquad 7 - 2 = 5 \\ i &= 5 \qquad 7 - 5 = 2 \end{aligned}$$

$$S = \prod_{3} (2, 5, 7)$$

$$S = (\overline{c} + b + \overline{a})\ (c + \overline{b} + a)\ (c + b + a)$$

La forma canónica producto de sumas se podría haber obtenido directamente de la tabla de verdad observando las combinaciones que hacen S = 0, y sustituyendo en cada una de ellas el valor 0 (cero) por una variable directa y el valor 1 por su expresión inversa.

	c	*b*	*a*	*S*
0	0	0	0	0
1	0	0	1	1
2	0	1	0	0
3	0	1	1	1
4	1	0	0	1
5	1	0	1	0
6	1	1	0	1
7	1	1	1	1

Tabla 12.15. *Tabla de verdad.*

12.7 Simplificación de funciones

En el apartado anterior se ha visto el proceso que hay que seguir para resolver los circuitos combinacionales. En primer lugar, se debe realizar la tabla de verdad y, a continuación, obtener la ecuación en forma canónica, bien en su primera forma, bien en su segunda. Una vez llegados a este punto, el siguiente paso consiste en simplificar, hasta donde sea posible, la ecuación resultante.

Existen varios métodos para simplificar las funciones booleanas, entre los que destacan los tres tipos que se describen a continuación.

A Método algebraico

Es un método que utiliza los postulados, las propiedades y los teoremas del álgebra de Boole.

Ejemplo 10

Dada la función $S = c \cdot b \cdot a + c \cdot \overline{b} \cdot a + \overline{c} \cdot b \cdot a$, se trata de simplificarla todo lo posible.

Solución

De los dos primeros términos se saca factor común *ca*:

$$S = c \cdot a \cdot (b + \overline{b}) + \overline{c} \cdot b \cdot a$$

como $(b + \overline{b}) = 1$ y $c \cdot a \cdot 1 = c \cdot a$, queda: $S = c \cdot a + \overline{c} \cdot b \cdot a$.

Se saca factor común *a* y se tiene: $S = a \cdot (c + \overline{c} \cdot b)$.

Por último, aplicando uno de los teoremas:

$$c + \overline{c}b = c + b$$

Por tanto, resulta: $S = a \cdot (c + b)$.

B Métodos numéricos

Son un conjunto de métodos, entre los que destaca el de Quine-McCluskey, en los que están basados los procesos de simplificación de funciones por medios informáticos.

C Método gráfico de Karnaugh

Esta herramienta para simplificar funciones es uno de los métodos más fáciles que existen si el número de variables que hay que simplificar no es muy elevado.

Normalmente se emplean hasta seis variables, pero en esta Unidad sólo analizaremos hasta cuatro variables.

Su fundamento se basa en la determinación, a partir de la tabla de verdad, de unas tablas denominadas **tablas de Karnaugh,** cuya forma depende del número de variables de entrada que se utilicen (Fig. 12.19).

El número dentro del recuadro indica el equivalente decimal de la combinación correspondiente. Hemos de insistir en que el lugar que ocupan estos números depende del peso que toman las variables, y que, en nuestro caso, son *d*, *c*, *b* y *a*, de mayor a menor peso.

En este método, los cuadrados correspondientes a los términos canónicos que forman parte de la función se indican mediante un 1, y los correspondientes a los

b \ a	0	1
0	0	1
1	2	3

c \ ab	00	01	11	10
0	0	2	3	1
1	4	6	7	5

cd \ ab	00	01	11	10
00	0	2	3	1
01	8	10	11	9
11	12	14	15	13
10	4	6	7	5

Fig. 12.19. *Tablas de Karnaugh para dos, tres y cuatro variables.*

términos que no forman parte de ella se dejan en blanco. En el caso de que existan combinaciones con términos indefinidos (X), estos términos se representarán como más interese, 1 o 0 (cero).

Para obtener la expresión más sencilla, es necesario realizar el mínimo número de agrupaciones con el mayor número de unos posibles, que formen parte de cuadros adyacentes. No hay que olvidar que la tabla es cerrada, es decir, que la columna (1, 9, 13, 5) es adyacente a la (0, 8, 12, 4) y la primera fila es adyacente a la última.

El procedimiento que se debe seguir para agrupar los unos será el siguiente:

1. Se toman todos los unos que no pueden formar parte de un grupo de dos por no ser adyacentes con ninguno.
2. Se forman los grupos de dos unos que no puedan formar parte de un grupo de cuatro.
3. Se toman los grupos de cuatro que no puedan formar parte de un grupo de ocho.
4. Cuando se cubran todos los unos, el proceso se detiene.
5. Se ha de tener en cuenta que un 1 puede estar incluido en tantos grupos como sea necesario.

Vamos a ver varios problemas de funciones lógicas utilizando las tablas de Karnaugh.

Ejemplo 11

Un circuito debe tener salida 1 cuando se cumpla la siguiente ecuación:

$$S = \sum_{3} (2, 6)$$

Solución

En la tabla o mapa de Karnaugh (Tabla 12.16) aparece la agrupación que hemos realizado.

En la agrupación debemos comprobar qué variable o variables cambian con el fin de eliminarlas.

En el mapa de la Tabla 12.16, cambia la variable *c*, por tanto, queda eliminada. La variable *a* queda en forma negada (0) y la *b* en forma directa (1).

$$S = b \cdot \overline{a}$$

c \ ab	00	01	11	10
0	0	1 (2)	3	1
1	4	1 (6)	7	5

Tabla 12.16. *Mapa de Karnaugh del Ejemplo 11.*

Ejemplo 12

Sea una función que responde a la ecuación:

$$S = \prod_{4} (0, 4, 8, 9, 10, 12)$$

Solución

Se plantea la tabla de Karnaugh (Tabla 12.17) y se realizan las agrupaciones que, en este caso, son tres:

Grupo 1: celdas 0, 4, 8, 12 $\rightarrow \overline{b} + \overline{a}$.

Grupo 2: celdas 8, 10 $\rightarrow d + \overline{c} + \overline{a}$.

Grupo 3: celdas 8, 9 $\rightarrow d + \overline{c} + \overline{b}$.

El resultado final será el producto de todas las sumas:

$$S = (\overline{b} + \overline{a})\ (d + \overline{c} + \overline{a})\ (d + \overline{c} + \overline{b})$$

cd \ ab	00	01	11	10
00	0 (0)	2	3	1
01	0 (8)	0 (10)	11	0 (9)
11	0 (12)	14	15	13
10	0 (4)	6	7	5

Tabla 12.17. *Mapa de Karnaugh del Ejemplo 12.*

Ejemplo 13

Una lámpara debe accionarse mediante la combinación de tres pulsadores (*c*, *b* y *a)*; cuando cumpla las siguientes condiciones:

1. Se accione un solo pulsador.
2. Se accionen dos pulsadores simultáneamente que no sean *a* y *b*.

Solución:

a) Representamos la tabla de verdad (Tabla 12.18).

b) Planteamos la resolución por minterms, o suma de productos.

La ecuación será: $S = \sum_3 (1, 2, 4, 5, 6)$, que, expresada en forma canónica, es: $S = \bar{c} \cdot \bar{b} \cdot a + \bar{c} \cdot b \cdot \bar{a} + c \cdot \bar{b} \cdot \bar{a} + c \cdot \bar{b} \cdot a + c \cdot b \cdot \bar{a}$

c) Se traslada al mapa de Karnaugh (Tabla 12.19) las salidas.

d) De la formación de los grupo y la simplificación se obtiene la ecuación:

$$S = c \cdot \bar{a} + b \cdot \bar{a} + \bar{b} \cdot a$$

e) La esquematización por contactos se muestra en la Figura 12.20.

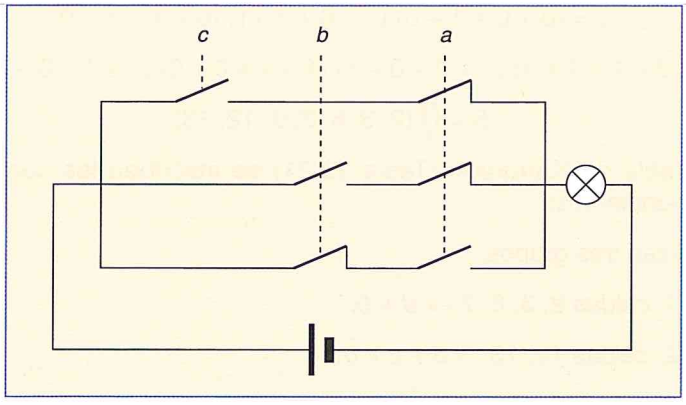


Fig. 12.20. *Representación eléctrica.*

f) La implantación por puertas lógicas se puede ver en la Figura 12.21.

Ejemplo 14

Ahora vamos a resolver el ejercicio anterior por maxterms o producto de sumas.

Solución

a) La tabla de verdad es la Tabla 12.18.

b) La resolución por productos de suma implica aplicar la fórmula explicada en el Apartado 12.6 y que se reproduce a continuación: $2^n - 1 - i$, donde *i* es el lugar que ocupan los términos de salida 0 y *n* el número de variables.

$$S = \prod_3 (0, 4, 7)$$

$$S = (\bar{c} + \bar{b} + \bar{a})\ (c + \bar{b} + \bar{a})\ (c + b + a)$$

c) Se traspasan estos resultados al mapa de Karnaugh (Tabla 12.20) y formamos los grupos:

Grupo 1: celdas 0, 4 → $\bar{b} + \bar{a}$.

	c	b	a	S
0	0	0	0	0
1	0	0	1	1
2	0	1	0	1
3	0	1	1	0
4	1	0	0	1
5	1	0	1	1
6	1	1	0	1
7	1	1	1	0

Tabla 12.18. *Tabla de verdad.*

c \ ab	00	01	11	10
0		1 (0, 2)		1 (3, 1)
1	1 (4)	1 (6)		1 (7, 5)

Tabla 12.19. *Mapa de Karnaugh del Ejemplo 13.*

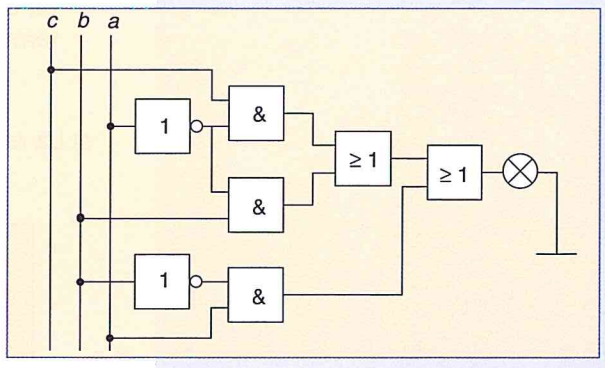


Fig. 12.21. *Representación por puertas. lógicas*

c \ ab	00	01	11	10
0	0 (0)			
1	0 (4)		0 (7)	

Tabla 12.20. *Mapa de Karnaugh del Ejemplo 14.*

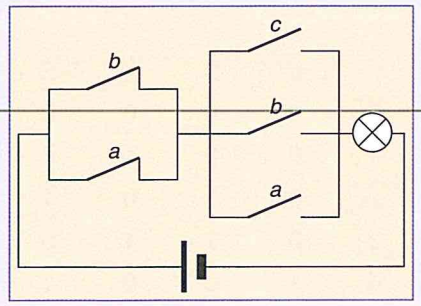


Fig. 12.22. *Representación eléctrica.*

ab / cd	00	01	11	10
00	0 (0)	0 (2)	0 (3)	(1)
01	(8)	(10)	(11)	0 (9)
11	0 (12)	(14)	(15)	0 (13)
10	(4)	0 (6)	0 (7)	(5)

Tabla 12.21.

Grupo 2: celda 7 → $c + b + a$.

La salida responderá al producto lógico de ambas expresiones:

$$S = (\overline{b} + \overline{a})\ (c + b + a)$$

d) La representación eléctrica la podemos observar en la Figura 12.22.

e) La implementación por cuentas lógicas se respresenta en la Figura 12.23.

Ejemplo 15

Dada una función de la forma:

$$S = (\overline{d} + \overline{c} + b + \overline{d})\ (\overline{d} + \overline{c} + b + a)\ (\overline{d} + c + b + \overline{a})$$

$$(\overline{d} + c + b + a)\ (d + \overline{c} + \overline{b} + a)\ (d + c + \overline{b} + \overline{a})\ (d + c + \overline{b} + a)$$

simplifica e implementa mediante puertas lógicas.

Solución

a) En primer lugar, se sustituyen dentro de cada suma las variables directas por 1 y las negadas por 0.

$$S = (0 + 0 + 1 + 0)\ (0 + 0 + 1 + 1)\ (0 + 1 + 1 + 0)$$

$$(0 + 1 + 1 + 1)\ (1 + 0 + 0 + 1)\ (1 + 1 + 0 + 0)\ (1 + 1 + 0 + 1)$$

$$S = \prod_{4} (2, 3, 6, 7, 9, 12, 13)$$

b) En la tabla de Karnaugh (Tabla 12.21) se inscriben las combinaciones correspondientes:

Se realizan tres grupos:

Grupo 1: celdas 2, 3, 6, 7 → $\overline{d} + b$.

Grupo 2: celdas 12, 13 → $d + c + \overline{b}$.

Grupo 3: celdas 9, 13 → $d + \overline{b} + a$.

La expresión final está compuesta por el producto de cada uno de los tres términos.

$$S = (\overline{d} + b)\ (d + c + \overline{b})\ (d + \overline{b} + a)$$

c) La realización mediante puertas lógicas es la de la Figura 12.23.

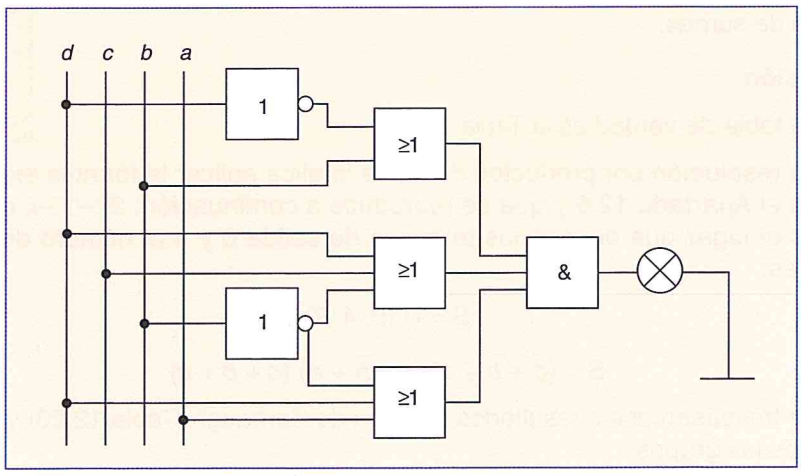


Fig. 12.23. *Representación por puertas lógicas.*

12.8 Implementación de funciones con puertas NAND y NOR

Las puertas NAND y NOR se conocen también como puertas universales debido a que todas las funciones lógicas se pueden construir con ellas.

Para poder realizar una función determinada o un circuito digital utilizando sólo puertas NAND o NOR, se deben aplicar los teoremas de Morgan tantas veces como sea necesario, hasta que toda la función se exprese en forma de productos o sumas negadas respectivamente.

Comenzaremos realizando las tres funciones básicas con puertas NAND y NOR de dos entradas, tal y como aparece en la Figura 12.24.

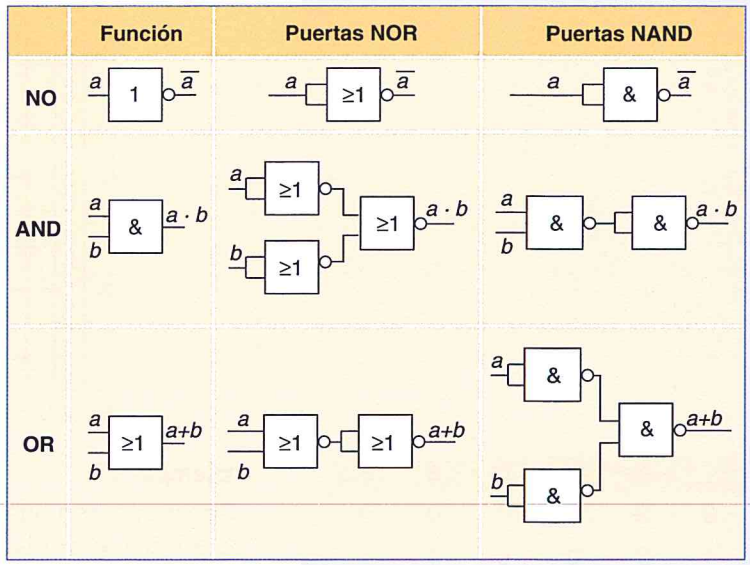


Fig. 12.24. *Funciones básicas realizadas con puertas NOR y NAND:*

Función inversión

Para realizar la función inversión con puertas NAND o NOR de dos entradas hay que recordar la tabla de verdad de este tipo de puertas. En ambas, cuando las dos entradas valen lo mismo, 1 o 0, la salida es 0 o 1 respectivamente.

- **Función AND**
  - Puertas NAND: $S = b \cdot a = \overline{\overline{b \cdot a}}$
  - Puertas NOR: $S = b \cdot a = \overline{\overline{b \cdot a}} = \overline{\overline{b} + \overline{a}}$
- **Función OR**
  - Puertas NAND: $S = b + a = \overline{\overline{b + a}} = \overline{\overline{b} \cdot \overline{a}}$
  - Puertas NOR: $S = b + a = \overline{\overline{b + a}}$

Ejemplo 16

Realiza un circuito que responda a la Tabla de verdad 12.22 con puertas NAND de dos entradas.

Solución

a) La salida responde a la siguiente ecuación, en primera forma canónica:

$$S = \sum_{3} (1, 2, 4, 5, 6, 7)$$

b) La simplificación por Karnaugh es la mostrada en la Tabla 12.23.

Realizamos tres grupos:

Grupo 1: celdas 4, 5, 6, 7 → *c*.

Grupo 2: celdas 2, 6 → $b \cdot \overline{a}$.

Grupo 3: celdas 1, 5 → $\overline{b} \cdot a$.

La expresión final estará formada por la suma lógica de los tres grupos.

$$S = c + b \cdot \overline{a} + \overline{b} \cdot a$$

c) Implementación por puertas NAND.

Se da a la función una doble inversión:

$$S = \overline{\overline{c + b \cdot \overline{a} + \overline{b} \cdot a}}$$

	c	*b*	*a*	*S*
0	0	0	0	0
1	0	0	1	1
2	0	1	0	1
3	0	1	1	0
4	1	0	0	1
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

Tabla 12.22. *Tabla de verdad del Ejemplo 16.*

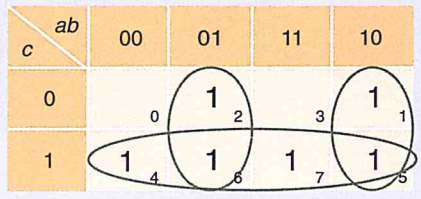


Tabla 12.23. *Mapa de Karnaugh del Ejemplo 16.*

Se opera una de las inversiones para transformar las sumas en productos.

$$S = \overline{\overline{c + b \cdot \bar{a} + \bar{b} \cdot a}} = \overline{\bar{c} \cdot \overline{b \cdot \bar{a}} \cdot \overline{\bar{b} \cdot a}}$$

d) La realización práctica con puertas NAND de dos entradas se muestra en la Figura 12.25.

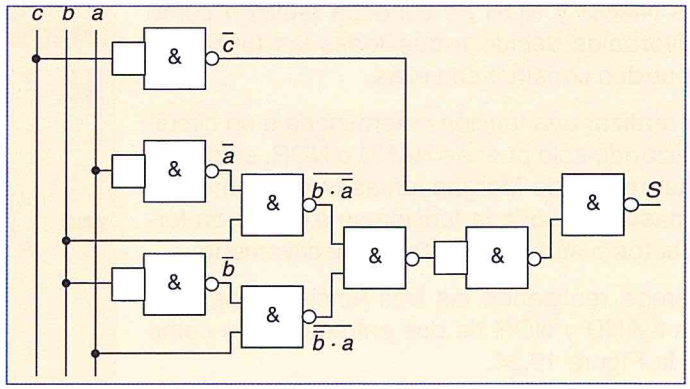


Fig. 12.25.

	d	c	b	a	S
0	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	0
3	0	0	1	1	1
4	0	1	0	0	0
5	0	1	0	1	1
6	0	1	1	0	0
7	0	1	1	1	1
8	1	0	0	0	1
9	1	0	0	1	1
10	1	0	1	0	0
11	1	0	1	1	1
12	1	1	0	0	0
13	1	1	0	1	0
14	1	1	1	0	X
15	1	1	1	1	X

Tabla 12.24. *Tabla de verdad del Ejemplo 17.*

cd \ ab	00	01	11	10
00	X 0	0 2	0 3	X 1
01	8	10	0 11	0 9
11	12	14	0 15	0 13
10	4	6	7	0 5

Tabla 12.25. *Mapa de Karnaugh del Ejemplo 17.*

Ejemplo 17

Construye, con puertas NOR de dos entradas, un circuito digital que responda a la Tabla de verdad 12.24.

Solución

a) En la tabla se puede observar que las combinaciones 14 y 15 son términos indefinidos, por lo que se pueden tomar como 1 o 0, según convenga.

b) Como el circuito se realizará con puertas NOR, se resuelve por la segunda forma canónica o producto de sumas, cuya ecuación será:

$$S = \prod_{4} (2, 3, 5, 9, 11, 13, 15) + \Phi(0, 1)$$

c) Se simplifica la tabla de Karnaugh (Tabla 12.25).

Se realizan tres grupos.

Grupo 1: celdas 0, 1, 2, 3 $\rightarrow (\bar{d} + \bar{c})$.

Grupo 2: celdas 1, 9, 13, 5 $\rightarrow (\bar{b} + a)$.

Grupo 3: celdas 9, 11, 13, 15 $\rightarrow (d + a)$.

El resultado final será el producto lógico de cada uno de los tres términos.

$$S = (\bar{d} + \bar{c})\ (\bar{b} + a)\ (d + a)$$

d) Se da a la función una doble inversión, y convertimos los productos en sumas.

$$S = \overline{\overline{(\bar{d} + \bar{c})\ (\bar{b} + a)\ (d + a)}} = \overline{\overline{\bar{d} + \bar{c} + \overline{\bar{b} + a} + \overline{d + a}}}$$

e) Se construye el circuito con puertas NOR de dos entradas (Fig. 12.26).

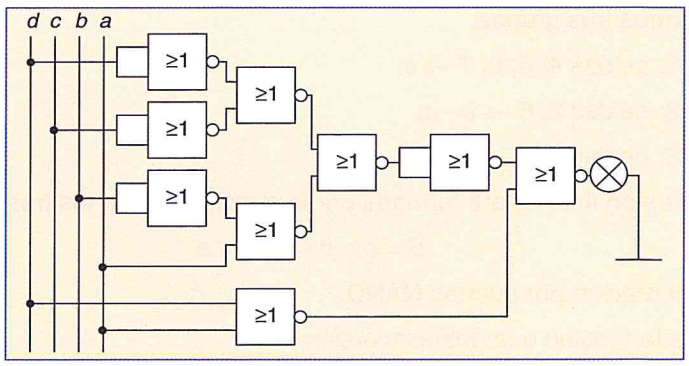


Fig. 12.26.

12.9 Circuitos combinacionales integrados

En los apartados anteriores hemos estudiado las características generales de un circuito digital y la manera de diseñarlo a partir de unas condiciones de funcionamiento, su tabla de verdad, la simplificación de la función y, además, la construcción de dicho circuito con puertas universales. A este tipo de circuito se le llama **circuito combinacional,** ya que en cada instante el estado lógico de sus salidas depende únicamente del estado de sus entradas, tal y como muestra la Figura 12.27.



Fig. 12.27. *Circuito combinacional.*

Ahora analizaremos unos circuitos según la función que realizan: no se hará su tabla de verdad ni el equivalente en puertas lógicas, sino que simplemente se utilizará el concepto de «caja negra» con unas entradas, unas salidas y, en caso necesario, unas entradas de control. Conociendo la misión que desempeñan, en el mercado se dispone de integrados que la realizan.

A Codificadores

Un **codificador** es un circuito combinacional que posee n salidas y 2^n entradas, de tal forma que, al accionarse una de sus entradas, en la salida aparece la combinación binaria correspondiente al número decimal asignado a esa entrada. El diagrama de bloques de un codificador aparece en la Figura 12.28. Los codificadores pueden ser sin prioridad o con prioridad.

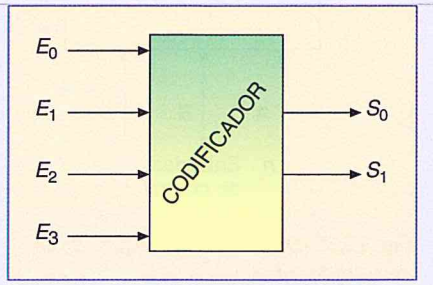


Fig. 12.28. *Diagrama de bloques de un codificador.*

En los **codificadores sin prioridad** no puede activarse más de una entrada al mismo tiempo y normalmente no se emplean.

Los más utilizados son los **codificadores con prioridad,** en los que si se produce una acción simultánea de varias de sus entradas, en la salida se presentará el código de aquella entrada que tenga asignada mayor peso significativo, que normalmente es la de mayor valor decimal.

En la Figura 12.29 se puede ver el codificador 74148, que tiene ocho líneas de entrada y tres líneas de salida.

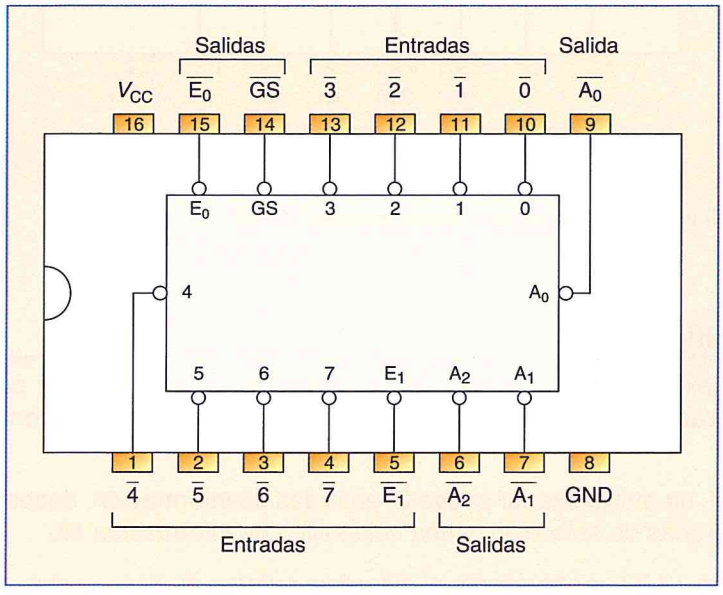


Fig. 12.29. *Codificador 74148.*

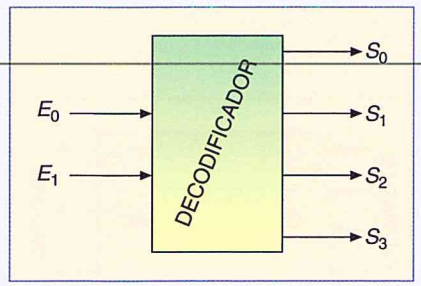


Fig. 12.30. *Decodificador.*

B Decodificadores

Un decodificador es un dispositivo combinacional que posee *n* entradas y un número de salidas menor o igual a 2^n, tal como se puede observar en la Figura 12.30. Básicamente, convierten información codificada (en cualquier tipo de código) en información sin codificar.

Los decodificadores se utilizan para dos aplicaciones, además de la ya citada, que son:

- Los demultiplexadores.
- La generación de funciones lógicas.

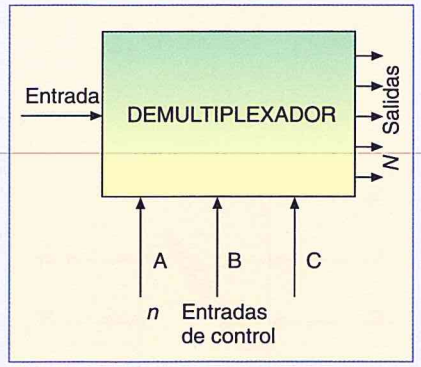


Fig. 12.31. *Diagrama de bloques de un demultiplexador.*

C Demultiplexador

Los **demultiplexadores** son circuitos lógicos combinacionales con una sola entrada, *N* salidas y *n* entradas de control, tal y como muestra la Figura 12.31. Su labor consiste en transmitir la información desde la entrada a la salida seleccionada mediante las entradas de control.

En el mercado hay circuitos integrados con la función decodificador/demultiplexador, tal es el caso del integrado 74138, que se puede apreciar en la Figura 12.32.

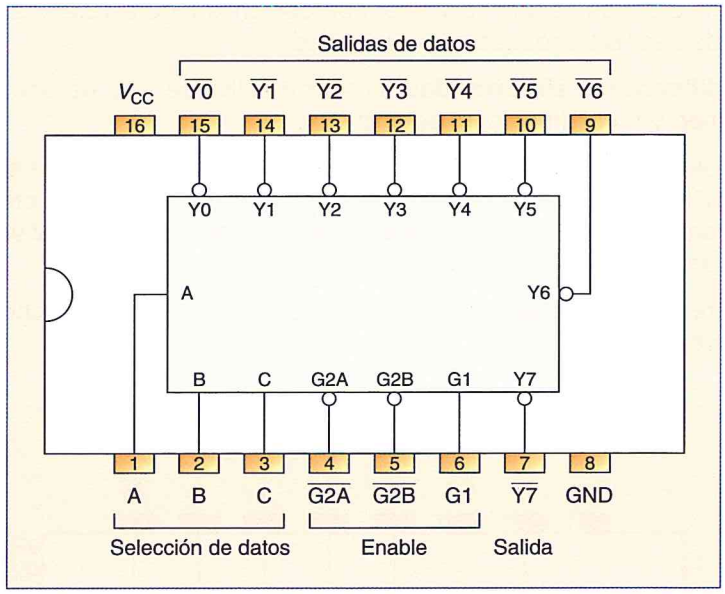


Fig. 12.32. *Decodificador demultiplexador. 74138.*

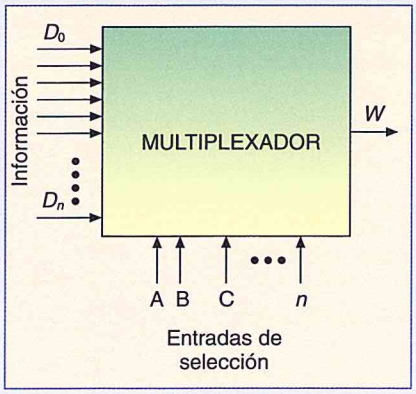


Fig. 12.33. *Multiplexador genérico.*

D Multiplexador

Un multiplexador es un circuito lógico combinacional cuya labor consiste en canalizar varias fuentes de información binaria hacia una línea común de salida.

En general, un multiplexador posee 2^n entradas de información, denominadas D_0 a D_n, *n* entradas de selección, y una sola salida de información *W*.

En la Figura 12.33 se representa el diagrama externo de conexiones de un multiplexador genérico.

El funcionamiento del multiplexador es el siguiente: cuando se presenta una combinación binaria en las entradas de selección, en la salida aparece un solo dato, correspondiente a la entrada que lleve asignada esta combinación binaria.

Existen a la venta varios circuitos integrados que hacen la función de multiplexar, por ejemplo, se dispone del multiplexador de ocho canales 74151 cuyo bloque funcional se representa en la Figura 12.34.

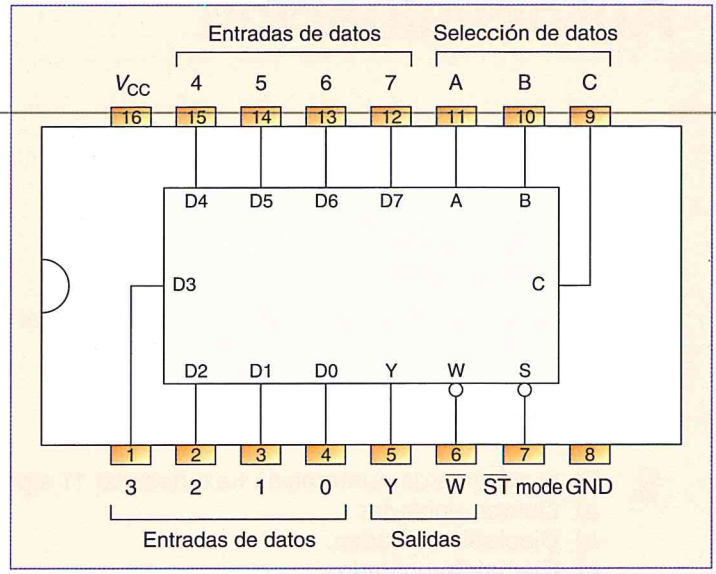


Fig. 12.34. *Decodificador multiplexador 74151.*

Comparador

Un comparador digital (Fig. 12.35), es un circuito lógico combinacional capaz de detectar las relaciones mayor (>), igual (=) y menor (<) entre dos configuraciones binarias.

En esencia, una comparación digital presenta:

- Dos grupos (A y B) de *n* líneas de entrada. Cada grupo de líneas canaliza hacia la entrada del comparador una palabra binaria de *n* bits.
- Tres líneas de salida. Al comparar las dos palabras binarias introducidas en el comparador, el sistema combinacional responderá activando una de las tres salidas siguientes.

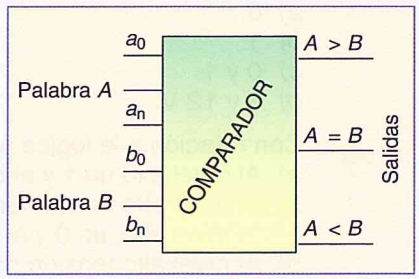


Fig. 12.35. *Estructura de comparador.*

A > B: cuando la palabra binaria *A* sea de magnitud superior a la *B*.

A = B: cuando la palabra binaria *A* sea igual a la *B*.

A < B: cuando la palabra binaria *A* sea de menor magnitud que la *B*.

Comercialmente se dispone de varios circuitos integrados que realizan la operación de comparar. En concreto, en la Figura 12.36 es el comparador 7485. Es un comparador de dos palabras de 4 bits, además dispone de tres entradas de propagación para poder conectarlo en cascada y obtener un comparador de 8 bits, si se agrupan dos y de 16 bits si se agrupan cuatro.

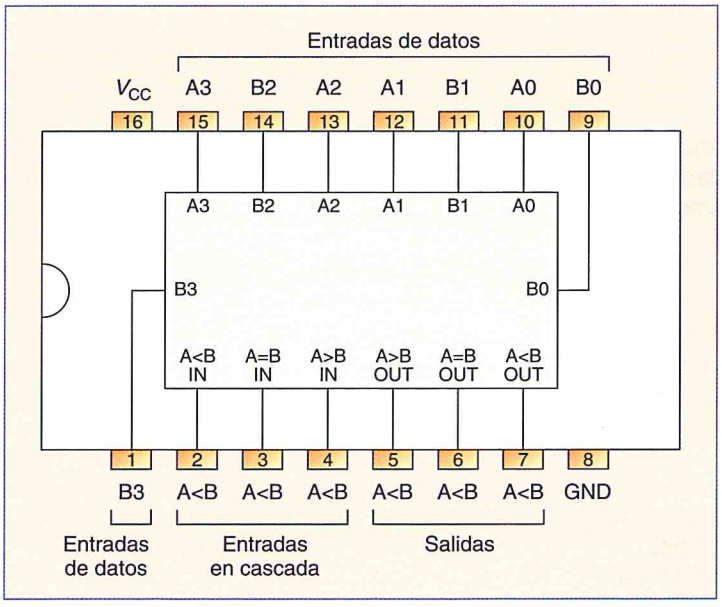


Fig. 12.36. *Diagrama de bloques de comparador. 7485.*

Autoevaluación

1 El número binario 1001 indica:
a) Diez unidades.
b) Nueve unidades.
c) Ocho unidades.
d) Siete unidades.

2 El número decimal 123 en el código BCD natural es:
a) 0000 1111 1001.
b) 0001 0110 1100.
c) 0001 0100 1100.
d) 0001 0010 0011.

3 En el sistema de numeración hexadecimal 11 significa:
a) Quince unidades.
b) Dieciséis unidades.
c) Diecisiete unidades.
d) Ninguna de las anteriores, pues en el sistema hexadecimal los números se representan con las letras de la A a la F.

4 ¿Cuáles pueden ser los valores de una señal en un circuito digital?
a) 0.
b) 1.
c) 0 y 1.
d) 5 y 12 V.

5 Con relación a la lógica positiva, se asigna:
a) Al nivel alto un 1 y al bajo un 0, o un 1 en función de la puerta utilizada.
b) Al nivel alto de tensión, un 1 y al bajo, un 0.
c) Al nivel alto, un 0 y al bajo, un 1.
d) Al nivel alto tensión positiva y al bajo tensión negativa.

6 La operación lógica AND entre dos variables da salida 1 solamente si:
a) Las dos entradas están a 1.
b) Las dos entradas están a 0.
c) La primera entrada está a 1 y la segunda a 0.
d) La primera entrada está a 0 y la segunda a 1.

7 ¿A qué tipo de puertas se les denomina universales?
a) A las AND, NOR, y NO.
b) A las NAND y las OR.
c) A las NAND y las NOR.
d) A las AND y OR.

8 ¿A qué conjunto de puertas elementales equivale una puerta NAND?
a) A un sumador en serie con un multiplicador.
b) A un multiplicador en serie con un inversor.
c) A un sumador en serie con un inversor.
d) A dos inversores en serie.

9 Uno de los teoremas de Morgan es:
a) $a + b = \overline{\overline{a + b}}$.
b) $a \cdot b = \overline{\overline{a \cdot b}}$.
c) $\overline{(a + b)} = \overline{\overline{a + b}}$.
d) $\overline{(a + b)} = \overline{a} \cdot \overline{b}$.

10 Para simplificar una función:
a) Ésta tiene que estar en producto de sumas.
b) Tiene que estar en la segunda forma canónica.
c) Puede estar en cualquiera de las formas canónicas.
d) Tiene que estar en suma de productos.

Cuestiones y ejercicios

1 Expresa los números 247, 512, 1024 y 914 en binario.

2 Expresa los números 1024 y 725 en hexadecimal, BCD natural con paridad par y BCD exceso a tres con paridad impar.

3 Expresa el número $1110001001010101_{(2)}$ en decimal y hexadecimal.

4 Expresa los números $FFF_{(H)}$ y $3\,FC4_{(H)}$ en decimal y binario.

5 Realiza las siguientes operaciones binarias:

a) 10010 + 1001

b) 110011 – 1011

6 Realiza la tabla de verdad de las siguientes funciones:

- $S = (a + b)$
- $S = a \cdot \overline{b} \cdot \overline{c} + a \cdot b \cdot c$
- $S = a \cdot \overline{b} + \overline{a} \cdot b$
- $S = \overline{a + b} + \ \cdot (\overline{a} + \overline{b})$

7 Un circuito lógico posee tres entradas *c*, *b*, *a* y una salida *S*. El circuito responderá 1 lógico cuando las entradas *c* y *b* sean 1 o cuando *b* y *a* valgan 0.

a) Determina la ecuación lógica sin simplificar en la forma de sumas de productos.

b) Expresa la ecuación lógica simplificada.

c) Dibuja el circuito eléctrico por contactos.

d) Dibuja el circuito utilizando puertas lógicas.

8 Un ascensor muestra la información de la planta en que se encuentra en la cabina como un número, codificando en binario de 4 dígitos. Se trata de realizar un sistema que avise cuando el ascensor esté en la planta 4, 6, 7, 8 y 12.

a) Calcula la tabla de verdad y la ecuación de un sistema lógico equivalente.

b) Simplifica el sistema mediante Karnaugh.

c) Realiza la ecuación simplificada con puertas AND, OR y NOT.

d) Determina la ecuación simplificada con puertas NAND de dos entradas.

9 Dada la función:

$F = c \cdot \overline{b} \cdot a + c \cdot b \cdot \overline{a} + c \cdot a + c \cdot \overline{b}$

a) Calcula la tabla de verdad y la ecuación del sistema.

b) Simplifica el sistema.

c) Realiza la ecuación simplificada con puertas NOR de dos entradas.

10 Dada la siguiente función:

$F = (c + a)\,(b + a)\,(\overline{c} + b)$

a) Calcula la tabla de verdad y la ecuación del sistema.

b) Simplifica el sistema.

c) Realiza la ecuación simplificada con puertas NAND de dos entradas.

11 Describe una puerta NOR de tres entradas, su tabla de verdad y la construcción con puertas NAND.

12 Examina el conexionado del circuito integrado (CI) 7404.

a) Monta el circuito 7404 en el panel del entrenador y aliméntalo a los 5 V necesarios.

b) Completa una tabla de la verdad; utiliza para ello un conmutador de estado a la entrada y un indicador a la salida.

13 Verifica las propiedades de la función lógica AND de dos entradas y la realización de puertas AND con varias entradas a partir de una de dos entradas.

Procedimiento:

a) Monta el CI 7408 en el entrenador y aliméntalo.

b) Comprueba por medio de conmutadores y visualizadores la tabla de la verdad de una puerta AND.

c) Realiza el montaje de una puerta AND de tres entradas utilizando el CI.

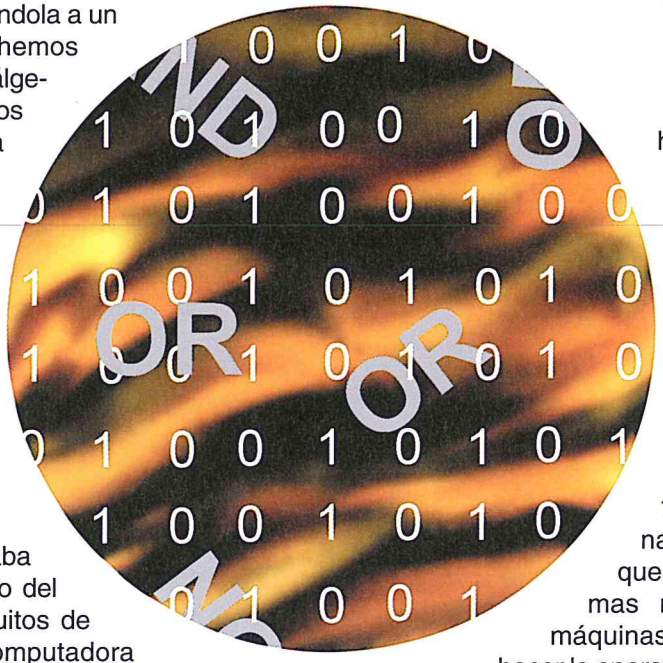
d) Conecta a una de las entradas un generador de onda cuadrada y la otra a un conmutador de estado. Estudia la salida en un osciloscopio cuando el conmutador está en estado alto y bajo. Realiza un cronograma de esta operación.

14 Repite la práctica anterior utilizando un CI 7432.

Tecnología y vida cotidiana

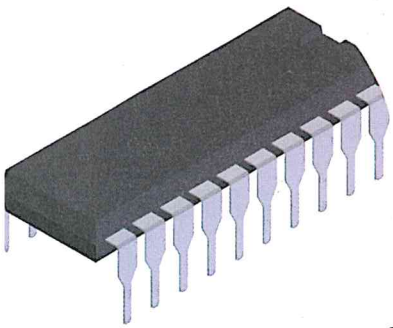
El camino hacia los circuitos digitales

George Boole publicó en 1854 *Las leyes del pensamiento,* libro sobre el que se basan los principios matemáticos que aplicamos a la Lógica y la Probabilidad. Por la vía de la matemática, Boole reorientó el estudio de la Lógica reduciéndola a un álgebra simple. Como ya hemos visto, en su aplicación del álgebra se recurre solamente a los valores binarios 1 y 0 y a tres operadores: AND (y), OR (o) y NOT (no).



No obstante, hubo que esperar hasta 1938 para encontrar la aplicación por la que el álgebra de Boole ha trascendido a la historia. Fue gracias al ingeniero electrotécnico y matemático Claude Shannon Elwood.

El trabajo de Shannon versaba sobre el análisis matemático del funcionamiento de los circuitos de relés, que llevaba una computadora mecánico-eléctrica construida en el MIT. El ingeniero reparó en que, si bien el analizador trabajaba de forma analógica, su comportamiento estaba gobernado en cualquier instante por la posición de los contactos de relés, y éstos siempre estaban en una de las dos posiciones: abiertos o cerrados *(on* u *off).* Esto fue lo que le hizo recordar el trabajo de George Boole. De ahí sólo quedaba un simple —pero muy importante— paso hacia el uso de circuitos de relés aplicado a la construcción de una «máquina lógica» digital, que pudiera llevar a efecto no sólo cálculos numéricos, sino otros tipos de «procesamiento de información».

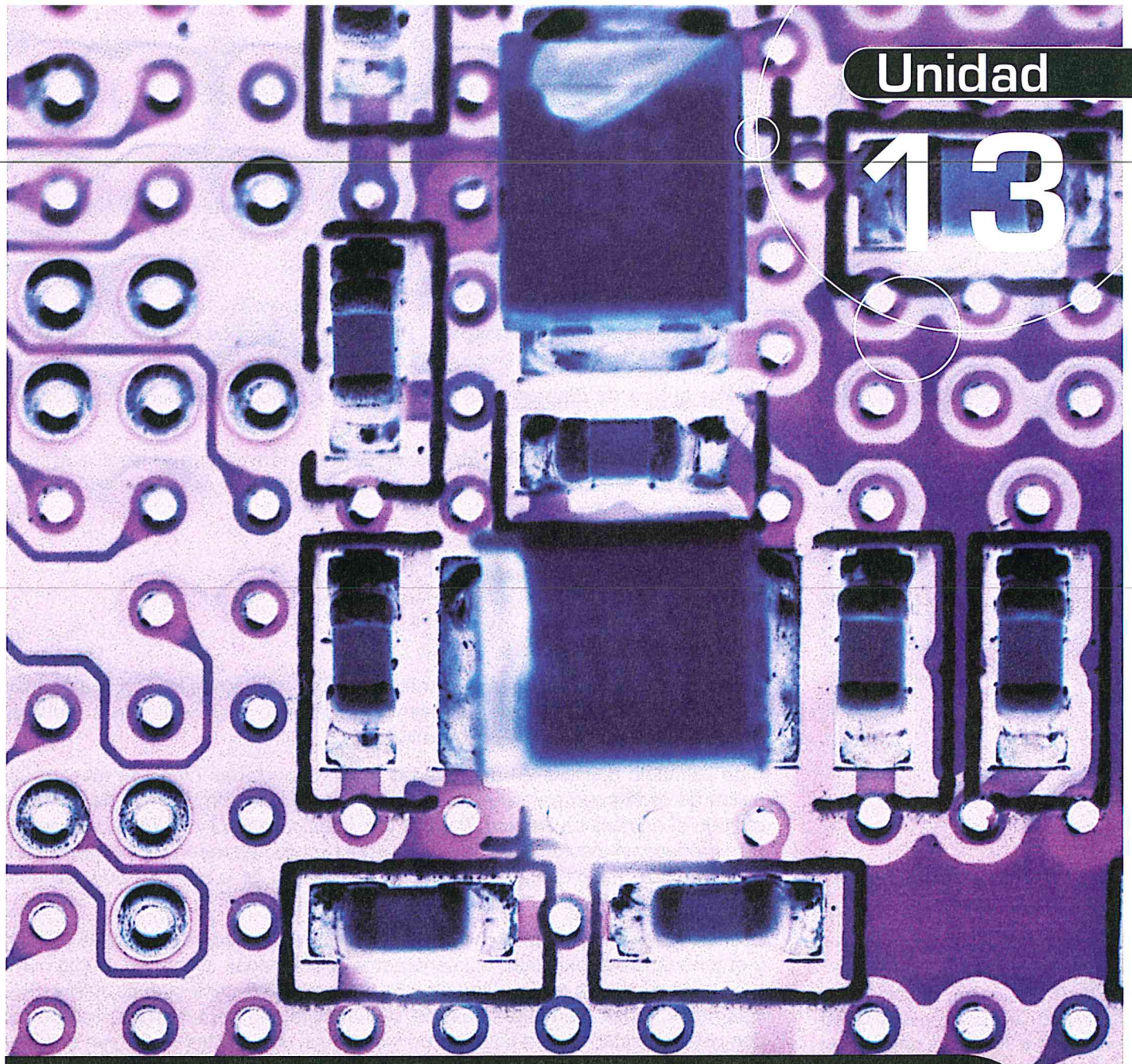


Esto pone de manifiesto la necesidad de un modelo matemático para interpretar la realidad que nos rodea. En ocasiones, aparece antes la teoría matemática, como ha sido este caso, que su aplicación práctica. Otras veces sucede al revés.

Así las cosas, ya solamente faltaba el desarrollo tecnológico. En efecto, con el álgebra de Boole sabemos hacer todo tipo de operaciones matemáticas con 0 y 1. El problema reside en que una persona tardará más en sumar 1 492 + 2 008 en el sistema binario que en el decimal. Cierto que, con el uso material de sistemas mecánico-eléctricos (antiguas máquinas calculadoras), sería posible hacer la operación más rápido. Pero aún no es suficiente.

El paso de gigante lo dio la electrónica con sus circuitos integrados, lo que fue posible gracias al desarrollo del transistor por parte de los investigadores de los Laboratorios Bell, John Bardeen, Walter Brattain y William Shockley, quienes recibieron el Nobel de Física en 1956 por esa investigación.

De modo que a finales de la década de 1960 apareció el circuito integrado (CI), que posibilitó la fabricación de varios transistores en un único sustrato de silicio en el que los cables de interconexión iban soldados. El circuito integrado permitió una posterior reducción del precio, el tamaño y los porcentajes de error. Este hito se debe a Jack Kilby, que presentó el primer chip en 1958, aunque no se reconocería su esfuerzo hasta el año 2000, cuando recibe el Nobel de Física. Ese año comparte el premio con Zhores Alferov y Herbert Kroemer, quienes desarrollaron los dispositivos semiconductores de alta velocidad, núcleo esencial de aparatos tan familiares como los CD o los lectores de códigos de barras.



Unidad 13

Circuitos secuenciales. Introducción al control cableado

El elemento fundamental de los circuitos secuenciales es la memoria, pues las salidas actuales dependen de la secuencia de estados anteriores. Es decir, han de recordar la historia previa. La memoria empleada en automatización industrial abarca desde los relés hasta los circuitos integrados de última generación. Aún, hoy en día, se siguen utilizando los esquemas realizados con relés para programar autómatas.